

ChordAI: A Locally-Hosted Generative System for Chord-Conditioned Songwriting

Presented by: Kjell-Inge Tech

Iver M. Mortensen
GitHub: @IverMortensen
E-post: imo059@uit.no

Hans Hjermundrud
GitHub: @HansHjermundrud
E-post: hhj012@uit.no

Theodor M. A Tredal
GitHub: @TheodorTredal
E-post: ttr042@uit.no

Eskil B. Heines
GitHub: @ebheines
E-post: ehe136@uit.no

October 1, 2026

[Demo-video](#) | [GitHub-repo](#)

Abstract

ChordAI is a fully local, open-source system that turns a natural-language song description into a complete song draft: a set of chord progressions, section-by-section lyrics, and an album cover. A user types something like “*I have this chorus for a 2000s pop song...*” and the system returns structured, playable output. The system is built around a small language model that acts as a *planner*, routing the request to three generative components: a chord-progression model we trained from scratch, a lyric generator, and a text-to-image model. The central research contribution is the chord model. We frame chord-progression generation as autoregressive token prediction conditioned on *genre* and *decade*, train it on the Chordonomicon dataset, and study it through a deliberate three-model progression, an LSTM baseline, a minimal transformer, and an aligned-batching transformer, evaluated with two complementary metrics. Our headline finding is non-monotone and, we argue, the most interesting result of the project: the naive transformer conditions on genre *less* strongly than the LSTM, because attention has the *option* to ignore the conditioning token whereas the LSTM is structurally forced to carry it; a corrected training recipe (document-aware batching plus prefix loss-masking) restores and then exceeds the LSTM’s conditioning fidelity while retaining the transformer’s predictive advantage. This is a finding about the *interaction* between architecture and training recipe, not simply “transformers beat RNNs.” Around this model we describe the lyric and image components, the planner, and the engineering that ties them into a single locally-hosted pipeline.

1 Introduction

1.1 Motivation

The project began with a concrete, personal frustration. When writing music on a guitar at an intermediate level, the hardest part is often simply *starting*: once a few chords are on the table, the creative process flows, but arriving at that first progression from a blank page is difficult. The original goal was therefore narrow, a tool that generates a chord progression to seed the creative process.

A bare chord generator felt too thin to stand on its own, so the scope grew deliberately and incrementally into a more complete songwriting assistant. If the system can produce chords, it can hand them to a lyric model and produce a full song; if it is producing a song, an album cover rounds out the artefact; and since typing chords in by hand is tedious, an audio component that detects chords from a recording removes that friction. A speech-to-text component was prototyped but ultimately dropped, it was not essential to the core experience, and time and compute had to be focused where they mattered most.

1.2 Scope and contributions

This report describes the system as a whole, but its weight is intentionally uneven, mirroring where the genuine generative-AI work lies:

- The **chord progression model** (Section 3) is the only generative component we *trained ourselves*. It is the centrepiece: a custom model, a designed experiment across three architectures, and a two-metric evaluation that produces a non-trivial finding.
- The **lyric generator** (Section 4) and **album-cover generator** (Section 5) build on strong pretrained models (Gemma 4 and Dreamshaper-XL). Our contribution there is integration and prompt engineering rather than training.
- The **planner** (Section 6) and the **server/client engineering** (Section 7) make the system usable. They are necessary and we describe the interesting decisions, but they are supporting features rather than the focus.
- The **audio chord-recognition** component (Section 8) is a separate input pathway with its own engineering narrative.

1.3 Design philosophy: fully local, fully open

A recurring theme is that everything runs locally on open-source models, with no calls to external commercial APIs. This was not the original plan. The first idea was an agent system built around a large hosted model (in the style of a Claude- or GPT-based orchestrator) using the Model Context Protocol (MCP) to connect to our individual model implementations. While working on the image model we realised the university cluster could host Gemma 4, a model large enough to act as a credible orchestrator and lyric writer. Once Gemma 4 was running, the fully-local route became feasible, and we dropped the MCP-based hosted-orchestrator design in favour of an entirely self-hosted, open-source pipeline. Keeping user data on our own infrastructure and avoiding external dependencies became an explicit design value.

2 System Overview

2.1 What the system does

The user describes a song in free text. The system interprets that description, generates candidate chord progressions, writes complete section-structured lyrics over those chords, and produces an album cover, then renders the result section by section in a chat-style interface. A representative output for the prompt “*I have this chorus for a 2000s pop song: F:maj D:min C:maj Bb:maj. Could you help me finish the song*” is a full pop song with intro, verses, pre-choruses, choruses, a bridge, and an outro, each labelled with its chord loop and lyrics, plus generated cover (appendix 6).

2.2 Architecture

The system has five components spanning three languages, summarised in Table 1.

Table 1: Components and technology stack.

Component	Location	Technology
Frontend	<code>client/</code>	Nuxt, Vue 3, TypeScript, Tailwind CSS
Backend server	<code>server/</code>	Go, Gin HTTP framework
Chord model	<code>chord-gen/</code>	Python, PyTorch, nanoGPT (custom GPT + LSTM)
Lyrics model	<code>lyric_generator_gemma4.py</code>	Python, Ollama (Gemma 4)
Image model	<code>image_generator.py</code>	Python, HuggingFace Diffusers (Dreamshaper-XL)
Planner	<code>server/system_connector/</code>	Go, Ollama (llama3.2)

The choice of Go for the orchestrating server is deliberate: it gives type safety, fast and clean subprocess management for invoking the Python model scripts, and good concurrency for streaming WebSocket progress. The generative models live in Python because they depend on PyTorch and Ollama; the Go server spawns them as subprocesses, which keeps the server itself lightweight and lets each model keep its own heavyweight dependency stack isolated.

2.3 Request flow

From prompt to result, the flow is:

1. The frontend opens a WebSocket to `/ws/generate` and sends a **PlannerInput** containing the free text and any optional fields (genre, decade, seed chords, ...).
2. The **system_connector** sends the free text to a local **llama3.2** model, which returns a structured **PlannerDecision**: genre, decade, tempo, vibe, the generation *mode* (generate / extend / section), any normalised seed chords, and an ordered *pipeline* of stages to run.
3. An **executor** runs each stage in pipeline order, streaming a **WSEvent** status update (**running** / **done** / **error**) to the frontend as each stage starts and finishes.
4. The chord model runs (Section 3), producing a *palette* of candidate progressions per section type.
5. The lyric model consumes that palette and writes the full lyrics.
6. The image model generates the cover.
7. The server assembles a **SongResult** and streams it as the final event; the frontend parses it and renders the song.

The data contracts that flow between components are small and explicit, which keeps the loosely-coupled, multi-language pipeline manageable: **PlannerInput** (what the frontend sends), **PlannerDecision** (what llama3.2 returns), **SongSpec** (the internal contract between the chord and lyric models), **SongResult** (what the frontend receives), and **WSEvent** (streaming progress).

3 The Chord Progression Model

This is the core of the project: the only generative model we trained ourselves, and the one component with a designed experiment and a quantitative evaluation behind it.

3.1 Problem formulation and the unifying insight

We set out to build a model that produces chord progressions conditioned on **main genre** and **decade**, supporting three use cases:

1. **Generate** - from genre + decade alone, produce a full progression.
2. **Extend** - given genre + decade + some seed chords, continue the progression.
3. **Section** - given a section (e.g. a verse), generate what comes next (e.g. a chorus).

The key conceptual insight is that these are *not* three models. They are the *same* autoregressive sequence model, differing only in *how much of the sequence is supplied as a prompt* at

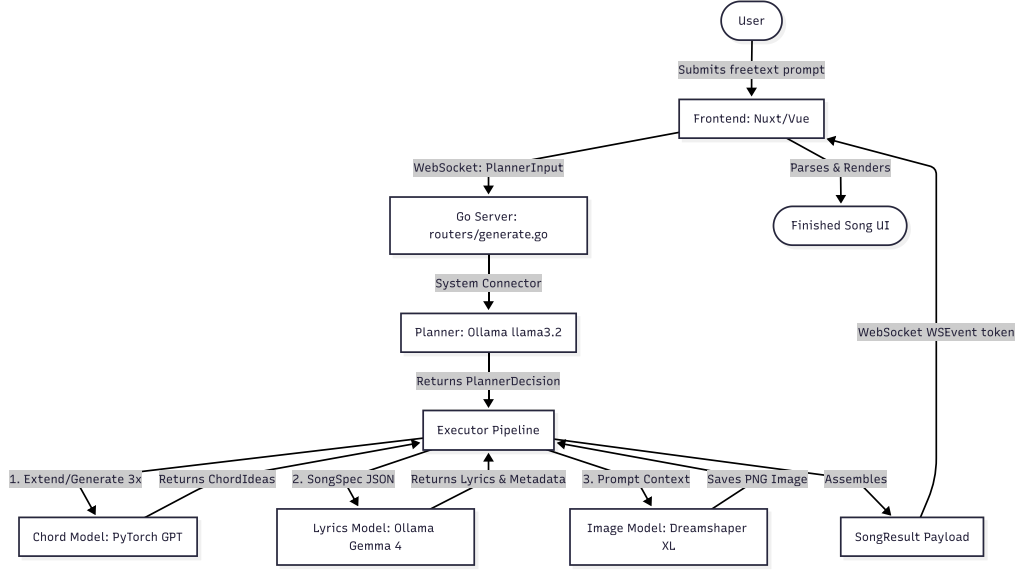


Figure 1: The ChordAI web interface: a chat-style client that shows stage-by-stage generation progress and renders the finished song with per-section chord headers and metadata pills.

inference time: empty chord context yields generation; partial chord context yields extension; context ending at a section boundary yields a new section. We train one network to perform a single task, predict the next token given the prefix, and the three use cases fall out of how much prefix we provide. This is the same trick that lets a single language model do completion, continuation, and infilling-by-prompting, applied to a musical token stream.

Each song is serialised as a single conditioning-prefixed token string:

```
<bos> <genre:pop> <decade:2010> <intro> C <verse> F C E7 Amin <chorus> F C G C ...
  ↪ <eos>
```

The `<bos>/<eos>` boundary tokens, with the conditioning prefix immediately after `<bos>`, are precisely what let the model learn that a fresh song begins with conditioning tokens — the mechanism that all three use cases rely on.

3.2 Dataset and cleaning

We use the Chordonomicon dataset ($\sim 680k$ songs).¹ Only three columns matter for our conditional-generation task: **chords** (a whitespace-separated progression with inline structure tags), **main_genre** (12 clean genre classes), and **decade**. All other columns (Spotify IDs, artist IDs, fine-grained sub-genres, etc.) are metadata and are dropped.

Evidence-driven cleaning. Before cleaning we ran SQL queries on the HuggingFace dataset console so that the cleaning decisions were driven by evidence rather than guesses. The findings that mattered:

- **Usable rows** (genre, decade, and chords all present): **301,564** of 680k. Genre is the binding constraint, only $\sim 352k$ rows have it, and every row with genre + decade also has chords.
- **Heavy genre imbalance:** pop (72.6k) down to electronic (2.0k), a $\sim 35\times$ spread across 12 classes.
- **Heavy decade imbalance:** the 2010s dominate (128.6k); everything before 1950 is negligible (1–83 rows per bucket) and contains data-entry errors.

¹<https://huggingface.co/datasets/ailsntua/Chordonomicon>

- **Small, light-tailed chord vocabulary:** 4,314 distinct chord strings, but only ~ 400 appear once and $\sim 1,180$ appear ≥ 100 times; the ≥ 100 set covers $>99\%$ of all chord-token occurrences. This is the empirical justification for *atomic* chord tokens rather than decomposing each chord into root + quality + bass.
- **Structure tags:** 8 semantic types (`intro`, `verse`, `chorus`, `bridge`, `outro`, `instrumental`, `interlude`, `solo`) but with numbering running absurdly high (`<verse_85>`); everything above `_5` is a sparse long tail.
- **Progression lengths** (chord tokens, excluding the conditioning prefix): median **77**, mean 81, p95 152, max 2,261 (a handful of pathologically repetitive songs).
- **Exact duplicates:** only 32 of 301k (0.01%), and inspection showed they are *genuinely common* simple loops (Amin F C G, Emin C G D) rather than ingestion errors, so **no deduplication was applied**.

Cleaning decisions. Table 2 lists each decision and its rationale. Every decision is exposed as a CLI flag so it can be swept for ablation without editing code.

Table 2: Data-cleaning decisions and rationale.

Decision	Choice	Why
Columns kept	<code>main_genre</code> , <code>decade</code> , <code>chords</code> only	Everything else is metadata irrelevant to conditioning.
Rows required	<code>genre</code> <i>and</i> <code>decade</code> <i>and</i> <code>chords</code> non-null	The premise is conditioning on genre + decade.
Pre-1950 rows	Dropped	$<2k$ rows total, several are data-entry errors.
Structure tags	Kept, numbering collapsed (<code><verse_2></code> $\rightarrow$ <code><verse></code>)	Tags carry real signal and enable the verse $\rightarrow$ chorus use case; numbering above <code>_5</code> is too sparse and ordinality is recoverable from position. $\rightarrow$ 8 clean section tokens.
Chord tokenisation	Atomic; vocab = chords seen $\geq 100\times$	$\sim 1,180$ chords cover $>99\%$ of chords in the dataset.
Rare chords ($<100\times$)	Replaced with <code><unk></code>	Honest probability mass in training; suppressed at sampling so output never contains <code><unk></code> .
Minimum length	Drop progressions <8 chord tokens	Costs only $\sim 1,460$ rows; shorter sequences cannot contain a meaningful progression.
Genre tokens	Slugified (<code>pop rock</code> $\rightarrow$ <code><genre:pop_rock></code>)	<i>Critical fix:</i> tokens with spaces get torn apart by <code>str.split()</code> downstream; all vocab tokens must be whitespace-free.
Conditioning encoding	Absolute tokens (<code><decade:2010></code>)	Simple; fits the token-stream model. Known limitation: decades are unordered, so sparse decades cannot borrow strength from neighbours.
Block size	256 tokens	Covers p95 (152) + prefix with headroom; truncating the rare 2,261-token outliers is cheaper than sizing for them (attention is quadratic in context).

Final data format. After cleaning: **$\sim 299,893$ songs** and a **vocabulary of 851 tokens** (4 special + 12 genre + ~ 8 decade + 8 section + ~ 819 chord). The split is *stratified by genre* into $\sim 269,905$ train / $\sim 29,988$ val ($\approx 23.16M$ / $2.57M$ tokens). Songs are concatenated into one stream per split with `<eos>`/`<bos>` as the only markers between them.

3.3 Pipeline and tooling

The data pipeline is three Python scripts plus the model code:

- `clean_chordonomicon.py` — raw CSV → cleaned parquet + `vocab.json` + `report.txt`. Applies every decision in Table 2.
- `prepare.py` — cleaned data → nanoGPT-format binaries (`train.bin`, `val.bin`, `meta.pkl`), plus `train.offsets.npy`/`val.offsets.npy` recording song-start positions (used by GPT v2).
- `train.py` — a single nanoGPT training script for both GPT variants, switched by a `doc_aware_batching` flag; `train_rnn.py` is the standalone LSTM trainer.
- `sample_chords.py` — adapted from nanoGPT’s sampler with word-level encode/decode, conditioning-based prompts, `<unk>`/`<bos>` suppression at sampling, and early stop at `<eos>`.
- `eval_nextchord.py` and `eval_conditioning.py` — the two evaluation scripts (Section 3.6), both variadic for N-model side-by-side comparison.
- `model_loader.py` — a shared dispatcher that loads GPT or RNN checkpoints uniformly (both expose the same `forward(idx, targets) → (logits, loss)` contract).

We chose **nanoGPT** (Karpathy) [9] for the transformer because it is small and simple to use out of the box. We also used nanoGPT in an assignment in the course so we were already familiar with it.

3.4 The three models

All three models use the *same* data, vocabulary, and stratified split. The progression is deliberate: **recurrent** → **transformer** → **aligned transformer**, and the three-way comparison is the central experimental result.

Model 0 — LSTM baseline. A standalone 2-layer LSTM ($n_embed=384$, $hidden=1024$, $n_layer=2$), sized to **~11.04M** parameters, intentionally matched to the transformer’s $\sim 10.95M$ so that a reviewer cannot attribute any difference to capacity. It uses v1-style random-window batching, no prefix masking, and no special conditioning mechanism: the conditioning tokens are simply leading tokens in the sequence, and the LSTM must carry that information forward in its hidden state through the entire song. This is the “what does a standard recurrent model achieve on this data?” reference, comparable to the recurrent family in the Chordonomicon paper [10].

GPT v1 — minimal-divergence transformer. Stock nanoGPT, **~10.95M** parameters (6 layers, 6 heads, n_embed 384, dropout 0.2, block size 256). It uses *random-window* batching: training windows are sampled at fully random positions in the concatenated token stream, so a window almost always begins mid-song. This creates the problem that motivates v2: because the median song (~ 79 tokens) is far shorter than the 256-token block, *most training windows do not contain the song’s <bos> <genre> <decade> prefix at all*. The model is therefore trained, in the majority of its examples, to predict chords *without the conditioning visible in context*. It still learns to use conditioning when it appears, but a large share of the training budget is spent on predictions where conditioning is simply absent, diluting the genre/decade signal.

GPT v2 — aligned-batching transformer. v2 is v1 plus two coupled changes, both controlled by the single `doc_aware_batching` flag; the model architecture (`model.py`) is *identical* to v1.

1. **Document-aware batching.** Training windows start at song boundaries (each `<bos>`), sampled from the offsets recorded by `prepare.py`. The conditioning prefix is therefore always at the front of every window. We use “aligned starts, windows run naturally”

(windows may run past `<eos>` into the next song rather than padding), which captures the conditioning-visible benefit without wasting $\sim 70\%$ of each window on padding.

2. **Token-identity prefix masking.** Every target whose token is a conditioning token (`<bos>`, `<genre:*>`, `<decade:*>`) is set to `-1`. nanoGPT’s loss already uses `ignore_index=-1`, so these positions are excluded automatically and `model.py` needed no change. The model is no longer graded on predicting tokens that are always *given* at inference; capacity is redirected to predicting chords and section tags.

A subtle but deliberate point: masking is by *token identity*, not position. A 256-token window holds ~ 3 songs, so a position-based “mask the first 3 tokens” rule would only catch the first song’s conditioning. Identity-based masking catches the conditioning tokens of *every* song in the window.

v2 was trained for 20k iterations *from scratch* (not fine-tuned from v1). Crucially, its raw validation loss is **not directly comparable** to v1’s, because v2 masks conditioning tokens out of its loss, the two are averaging different sets of positions. This non-comparability is the specific reason we built the two common-yardstick evaluation metrics in Section 3.6.

3.5 Inference: one model, three behaviours

All three models support the three use cases via the prompt-length trick of Section 3.1:

- **Generate** — prompt `<bos> <genre> <decade>`, sample until `<eos>`.
- **Extend** — the same prompt plus a partial chord sequence.
- **Section** — a prompt ending at (or appending) a `<chorus>` etc. token.

`sample_chords.py` implements these as three CLI modes and masks `<unk>` and `<bos>` from the sampler’s logits so generated output never contains them.

3.6 Evaluation: two metrics, because one is not enough

The claim under test is “v2 strengthens conditioning, and architecture matters.” A single metric cannot establish this, because the most obvious metric, next-chord prediction, is structurally insensitive to the very thing v2 changes. We therefore use two complementary metrics.

Metric 1: next-chord prediction (`eval_nextchord.py`). Teacher-forced top-1 / top-3 next-chord accuracy on the validation set. For each position in each val song we feed the model the *true* preceding context (genre, decade, and all real chords so far), take its prediction, and compare to the true next chord. Only **chord tokens** count in the denominator (structure tags, conditioning, and specials excluded), since “can it predict the next chord” is the musical question. This is the Chordonomicon paper’s metric. A caveat is baked into its design: because the true local chord context is always present, next-chord prediction is dominantly a *local* task, given **C G Amin**, the next chord is **F** in almost any genre. This is the regime *least* likely to reveal a conditioning difference, which is exactly why Metric 2 exists.

Metric 2: conditioning sensitivity (`eval_conditioning.py`). The complement: *free generation* from `<bos> <genre> <decade>` alone, with no chord context to lean on, so conditioning is the only available signal. For each genre we generate $N=300$ progressions (decade and temperature held fixed across models to isolate genre and match sampling randomness), build a normalised chord histogram per genre, and compute the **mean pairwise Jensen–Shannon divergence (JSD)** across the 12 genres as a single “genre-separation” score per model. With $M = (P + Q)/2$,

$$\text{JSD}(P \parallel Q) = \frac{1}{2} \text{KL}(P \parallel M) + \frac{1}{2} \text{KL}(Q \parallel M), \quad \overline{\text{JSD}} = \frac{2}{G(G-1)} \sum_{i < j} \text{JSD}(p_i \parallel p_j),$$

where p_i is genre i 's chord histogram and $G = 12$. JSD is bounded in $[0, 1]$; higher means the genres are more distinct in the model's output. A model that genuinely uses the genre token produces well-separated per-genre distributions; one that ignores it produces the same chord soup regardless. The script also reports musically interpretable per-genre chord-feature rates, the share of seventh / diminished / augmented / extended / slash chords, so that "jazz generates more 7ths than punk" becomes direct, human-verifiable evidence of conditioning.

3.7 Results

All numbers below come from runs on the trained checkpoints with identical evaluation procedures.

Table 3: Aggregate next-chord prediction accuracy (chord tokens only).

Model	top-1	top-3
RNN (Model 0)	74.83%	89.47%
GPT v1 (minimal-div)	78.67%	91.01%
GPT v2 (aligned)	78.73%	91.01%

Next-chord prediction (aggregate). Two findings. **(a) Architecture matters:** the transformers beat the RNN by ~ 4 points top-1, with a consistent ~ 1.5 -point gap on top-3k, a real, substantial, defensible result. **(b) v1 and v2 are essentially indistinguishable** on this metric (~ 0.06 points), exactly as anticipated, because v2's changes target conditioning, not local next-chord prediction.

Next-chord prediction (per genre). The transformers' advantage over the RNN is *not* uniform (Table 4).

Table 4: Per-genre next-chord top-1 advantage of GPT v1 over the RNN. v2's deltas are within ~ 0.1 pt of v1's per genre.

Genre	Δ top-1	Genre	Δ top-1
jazz	+ 6.86	punk	+3.13
soul	+ 6.02	country	+2.95
metal	+4.96	rap	+2.33
rock	+4.91	reggae	+1.75
pop rock	+4.41	electronic	+1.24
pop	+3.44		

The gap *grows* for harmonically complex / less locally predictable genres (jazz, soul, rock-family) and *shrinks* for simple / repetitive ones (rap, reggae, electronic, where "the loop continues" carries you a long way). This is exactly where attention's mechanism predicts it should win: when the model must integrate information across the context rather than continue a local pattern, attention's direct access beats the LSTM's compressed hidden state; for simple, loop-dominated genres the LSTM's local machinery is nearly enough.

Conditioning sensitivity (the interesting result). The ordering is **v2 > RNN > v1**, three-for-three, with the RNN consistently *between* the two transformers. This is the most intellectually interesting finding of the project, and it took the three-way comparison to reveal it: it is explicitly *not* a monotone "newer is better" progression. The naive transformer conditions *less* strongly than the LSTM, and v2's training fix is what restores and then exceeds the RNN's conditioning fidelity.

Table 5: Mean pairwise JSD genre-separation score (higher = stronger conditioning). The ordering **v2** > **RNN** > **v1** holds across all three conditions.

Condition	RNN	v1	v2
decade 2010, $T=1.0$	0.0800	0.0739	0.0898
decade 1990, $T=1.0$	0.0942	0.0881	0.1101
decade 2010, $T=1.2$	0.0729	0.0676	0.0798

Why RNN > v1 (the architectural explanation). The result becomes intuitive once one asks what each model is *forced* to do with the conditioning token. The **RNN**, by architectural necessity, must carry the conditioning through its hidden state at every step, there is no other path by which genre/decade can reach later positions once they have entered the input. The long-range weakness of recurrent nets is real, but here it doubles as a *forcing function*: the LSTM must fold conditioning into its running representation or it cannot use it at all. The **v1 transformer**, with attention, has the *option* to ignore the conditioning token: it is one token, often far back, and (the dilution problem) most training windows did not even contain it. Attention’s flexibility worked *against* the conditioning task, without pressure to attend to the conditioning, v1 learned to mostly not bother. **v2’s batching change applies the pressure the RNN got for free**: every window starts with the conditioning, and prefix masking removes the trivial reward of predicting the given conditioning tokens, so the loss now comes entirely from predicting chords and sections *given* conditioning visible at position 0. The transformer’s attention, now reliably exposed to conditioning in training, can out-perform the RNN’s hidden-state bottleneck.

The result in musical terms. JSD is abstract; chord-feature rates make the effect concrete (Table 6).

Table 6: Selected chord-feature rates (% of generated chords) at decade 1990, $T=1.0$.

Feature, genre	RNN	v1	v2
7th, jazz	20.1	16.1	27.5
7th, soul	30.8	19.1	25.5
7th, metal	2.5	1.7	1.3
7th, punk	2.2	1.1	1.4
ext, jazz	3.8	4.2	5.6
ext, soul	6.0	5.7	5.6
ext, punk	1.0	0.9	1.4

Two observations. First, *all three* models keep harmonically simple genres (metal, punk) simple, none suddenly emits 25% seventh chords in punk, so the conditioning signal is directionally correct everywhere. Second, for the harmonically rich genres (jazz, soul), v1’s expressiveness is visibly *suppressed* relative to both the RNN and v2, direct evidence of conditioning dilution, and v2 restores, and in jazz exceeds, the RNN’s commitment to genre-characteristic harmony.

Secondary findings. All three models hit <eos> in 95–100% of free generations at $T=1.0$, producing complete, well-terminated songs; the “looping problem” seen at $T=0.5$ in early sampling, where the same chords appear again and again, was a low-temperature degenerate-sampling artefact, not a fundamental flaw, the practical recommendation is $T \approx 1.0$. Generated lengths (mean ~ 70 –94 tokens) closely match the training median of 77. Raising temperature from 1.0 to 1.2 lowers JSD uniformly for all three models (by ~ 0.007 each), blurring genre identity but *preserving* the v2 > RNN > v1 ordering.

3.8 The headline conclusion (and how we scope it)

The full two-metric, three-model picture supports a precise, defensible claim:

Attention substantially improves next-chord prediction over recurrence by ~ 4 aggregate points and ~ 7 points on the most harmonically complex genres. But on conditioning faithfulness in free generation the picture is more nuanced: a naive transformer (v1) conditions less strongly than an LSTM, because attention has the option to ignore the conditioning token while an LSTM is structurally forced to carry it. Document-aware batching and prefix masking (v2) apply to the transformer the same pressure the RNN gets for free, producing a model that has the transformer’s predictive advantage and exceeds the RNN’s conditioning fidelity.

This is a finding about the *interaction* between architecture and training recipe, not just “transformers beat RNNs”, and it required both metrics and all three models to see. We scope the individual claims honestly: the transformer-over-RNN next-chord gap is large and unambiguous ($\sim 2.3\text{M}$ chord predictions, ~ 4 -point aggregate gap, consistent per-genre pattern); the $v2 > v1$ conditioning improvement is robust (three conditions, all positive, largest $+18\%$ relative); the $v2 > \text{RNN}$ gap is modest but reproduces across all three conditions (mean $+0.0108$); and the $\text{RNN} > v1$ gap, though small ($\sim +0.006$), reproduces almost identically across conditions, which makes it look more like signal than noise.

3.9 Limitations

These are deliberate and documented:

- **No prefix loss-masking for v1** — by design, since spending capacity on conditioning-token prediction is part of what makes v1 the honest baseline.
- **Absolute (unordered) decade tokens** — sparse decades cannot share strength with neighbours; ordinal/relative encoding is plausible future work.
- **Aligned-starts batching in v2, not one-song-per-window** — windows can still run past `<eos>`; token-identity masking handles the loss cleanly, but the model can still attend across song boundaries. Full block-diagonal document-isolation attention is the textbook-correct extension.
- **Single-seed evaluation** — robustness was established by varying *conditions* (decades, temperatures) rather than seeds. This is arguably stronger for directional claims but gives no formal error bars; multi-seed runs would.
- **No class-balancing** — the $35\times$ genre imbalance was not addressed by reweighting; per-genre evaluation cuts mitigate this but it remains a plausible improvement.

4 Lyric Generation

The lyric generator turns the chord model’s output into complete, section-structured lyrics. It is an integration-and-prompt-engineering contribution rather than a trained model, and is a supporting feature of the wider system rather than its central focus.

4.1 From OLMo-2 to Gemma 4

The first version loaded `allenai/OLMo-2-1124-7B-Instruct` [6] directly via `HuggingFace transformers` [7], generating in a single pass from verse and chorus chords, a genre, and a language. Its limitations were largely structural: chords were split into only two sections (no intros, bridges, or outros); there was no awareness of era, tempo, or harmonic mood; and the model was tightly coupled to OLMo-2, making it hard to swap. We initially assumed that a comparatively small model would suffice, but the output fell well short of the quality our

application required (see Section 4.3), prompting a move to the larger Gemma 4 [5] together with a more elaborate prompting strategy.

The improved version runs Gemma 4 via Ollama [8] as one stage in the pipeline. It consumes the chord model’s token string directly, parsing genre and decade and producing a labelled block of lyrics for every section; it interprets the harmonic mood, era, and tempo of the chords into emotional and stylistic guidance for the prompt; and it supports a free-text “vibe” parameter and per-genre style guides. Both a `.txt` and a `.json SongSpec` are emitted for downstream use, and a `-model` flag removes the hard OLMo-2 coupling, with the CLI mirroring `sample_chords.py` so a single orchestrator can drive both stages.

4.2 The chord palette and prompt rules

Two refinements to how chords reach the lyric model improved output quality. First, rather than passing a single sampler run verbatim, which could propagate a long repetitive loop, or a single malformed run, into the whole song, we run the chord model three times per request and merge the results into a *ChordIdeas palette*, reducing each progression to its shortest repeating cycle (`detectCycle`) and deduplicating across runs. Gemma is then given the palette as creative material and the freedom to design the song structure itself. Second, to stop the model prepending chord symbols to individual lyric lines, a habit drawn from the chord/lyric sheets in its training data. We added explicit formatting rules that confine chords to a single per-section annotation and cap the chord count per section.

4.3 Comparison of the first and second models

The difference in lyric quality between the two models is clear. The first model produced simplistic, generic phrasing with little of the figurative depth of genuine lyrics:

```
Life’s a rhythm, we’re dancing to,  
On this journey, just me and you,  
Chasing horizons, under the blue,  
With every step, I feel anew
```

The lines lean on stock phrases (“chasing horizons”, “come what may”) and reach only for the most predictable rhymes, reading as assembled rather than authored.

By contrast, Gemma 4 produced more vivid and specific imagery and sustained a coherent theme across the song:

```
Streetlights bleed across the pavement now  
Electric dreams we never knew somehow  
Every beat is pulling us along the wire  
Caught in this moment, fueled by neon fire
```

Stronger verbs (“bleed”, “pulling us along the wire”) and concrete images give the lines a distinct voice, and the song holds to a single neon, late-night conceit throughout — qualities largely absent from the first model’s output.

5 Album Cover Generation

To generate the album art, the pipeline implements a text-to-image synthesis approach developed within the **PyTorch** ecosystem, leveraging the Hugging Face `diffusers` library [13]. The system utilizes `Lykon/dreamshaper-xl-v2-turbo`, a pre-trained latent diffusion model. The model is a fine-tuned variant of the foundational `stabilityai/stable-diffusion-xl-base-1.0` (SDXL) architecture, optimized to deliver high-quality, stylized imagery at a native resolution of 1024×1024 pixels using significantly fewer inference steps.

While training a generative model from scratch such as a Generative Adversarial Network (GAN) was considered, it was deemed impractical for this project. Developing a custom GAN requires extensive GPU cluster resources and long training time. Furthermore, GANs are notoriously difficult to optimize due to inherent training instabilities, non-convergence, and the risk of mode collapse. Consequently, leveraging a state-of-the-art pre-trained diffusion model provided a far more computationally efficient and practical solution for generating high-fidelity art [14].

Latent Diffusion Foundations: Instead of operating directly in high-dimensional pixel space, the model performs the forward and reverse diffusion processes within a compressed, lower-dimensional latent space managed by an Autoencoder [12].

Turbo/distillation advantage: This specific checkpoint from Lykon/*dreamshaper-xl-v2-turbo* incorporates Adversarial Diffusion Distillation (ADD). This allows the model to bypass the traditional 30-50 inference steps, generating high-fidelity images in just 8 to 10 steps while maintaining high structural integrity [11].

Hardware Optimization & Memory Management: To ensure stable execution on shared server hardware and prevent Out-of-Memory (OOM) errors, several PyTorch-centric memory optimizations were implemented. The pipeline utilizes half-precision floating-point format (`torch.float16`), effectively halving the VRAM footprint. Additionally, sequential CPU offloading is enabled, dynamically routing sub-modules to the active device backend (e.g., NVIDIA CUDA or Apple Silicon MPS) only during active tensor computation. Finally, VAE slicing is applied to decode the latent tensor in smaller spatial batches, flattening peak memory spikes during pixel reconstruction.

Input Conditioning & Prompt Engineering The image generator implements programmatic prompt modification to the user query to enforce strict quality controls and aesthetic boundaries, this includes a hard-coded **Enhanced Prompt** and a **Negative Prompt**.

- **Enhanced Prompt** Appends hardcoded quality modifiers (e.g., "**breathhtaking conceptual art**", "**divine lighting**") to the user's input. This anchors the latent space trajectory toward high-aesthetic regions of the model's text-embed space.
- **Negative Prompting:** Explicitly guides the U-Net away from specific latent characteristics. In this implementation, it tries to actively penalize representations of human anatomy, text artifacts, and poor photographic composition (e.g., "**washed out colors**", "**bad anatomy**").

The generation execution uses the following hyperparameters via the Hugging Face `diffusers` library:

Analysis of Generated Outputs: As demonstrated in Figure 2, the pipeline successfully translates diverse genre-specific prompts into highly coherent, stylized album art. The differences between the alternative indie rock aesthetic in Figure 2a and the dark, atmospheric metal theme in Figure 2b reflect how distinct textual conditioning navigates the model toward separate high-aesthetic clusters within the latent space.

Furthermore, the visual outputs provide an empirical validation for the implemented prompt engineering strategy. The influence of the hardcoded quality modifiers is evident in the dramatic, professional-grade lighting and textured depth across both compositions. Concurrently, the total absence of distorted human features, malformed text, or cluttered photographic artifacts confirms the efficacy of the comprehensive negative prompt string, ensuring the output aligns with the aesthetic hard coded negative and enhanced prompts.

Table 7: Inference Hyperparameters and Technical Justifications

Parameter	Value	Technical Justification
num_inference_steps	10	Aligns with the ADD/Turbo distillation mechanics to maximize speed without degrading visual coherence.
guidance_scale (CFG)	2.0	Lower Classifier-Free Guidance scale is typical for Turbo/LCM models; prevents oversaturation while adhering to the prompt.
width / height	1024 × 1024	Matches the native training resolution of SDXL, preventing aspect-ratio distortion and duplication artifacts.



(a) Album cover generated with prompt: An indie rock song like Arctic monkeys from 2005



(b) Album cover generated with prompt: a heavy metal song from the 90's in c minor

Figure 2: Examples of system-generated album artwork based on textual conditioning.

6 The Planner

The planner (`system_connector`) is the AI routing layer that makes free-text input possible. It sends the user’s text to a local **llama3.2** [8] via the Ollama REST API with a system prompt instructing it to return a JSON `PlannerDecision`: the musical parameters (genre, decade, tempo, vibe), the generation *mode*, any seed chords, and the ordered pipeline of stages to run. If llama3.2 returns malformed JSON, the planner retries once with a correction prompt. A small, fast local model is well-suited here: structured parameter extraction is undemanding, and keeping it local avoids sending user input to an external service which is consistent with the fully-local design value.

The planner also absorbs two pieces of responsibility that prevent downstream failures:

- **Genre mapping.** The planner may emit genre names (`rnb`, `hiphop`, `edm`, `blues`, `folk`) that are not in the chord model’s vocabulary; a lookup table maps these to the nearest training genre (`soul`, `rap`, `electronic`, `rock`, `alternative`). Decades outside the trained 1950–2020 range are clamped to the nearest boundary.
- **Seed-chord normalisation.** The planner is told the vocabulary’s notation rules (sharps as `s`, flats as `b`, minor as a `min` suffix, major as no suffix) and instructed to normalise the

user’s chords and to prepend a section tag when the user names a section (“these are my chorus chords” → `<chorus> F G E7 Amin`). See Section 7 for the defence-in-depth that backs this up.

7 System Integration and Engineering

This section covers the supporting full-stack and tooling work. It is deliberately brief: it is necessary for a usable, reproducible system but is not the generative-AI focus of the report.

7.1 Server and client

The Go [1]/Gin [2] server exposes three endpoints:

- `GET /api/ping` - a connectivity check;
- `POST /api/generate` - blocking REST generation;
- `GET /ws/generate` - WebSocket streaming, the primary transport.

WebSocket is preferred because it lets the UI show stage-by-stage progress (Planner → Chords → Lyrics), with REST kept as an automatic fallback. The `executor` iterates the planner’s pipeline and emits a `WSEvent` as each stage starts and finishes. One subtle but important detail: the Python subprocess’s `stdout` is routed to the server’s `stderr` so that rich console markup never leaks into the API response, and the lyric runner reads its clean output from the JSON sidecar rather than from captured console text.

The Nuxt [4]/Vue [3] client is a chat-style single-page application (Figure 3). A `useGenerate` composable manages the WebSocket and falls back to REST on failure, a `StatusBar` shows live stage progress, and a `SongOutput` component parses the lyrics into sections and renders metadata pills (genre, decade, BPM, vibe) alongside colour-coded chord pills. As a deliberate design choice that mirrors the lyric model’s formatting rules, chords are shown once per section as a styled header rather than inline between lyric lines, and streaming text is hidden until the formatted result is complete.

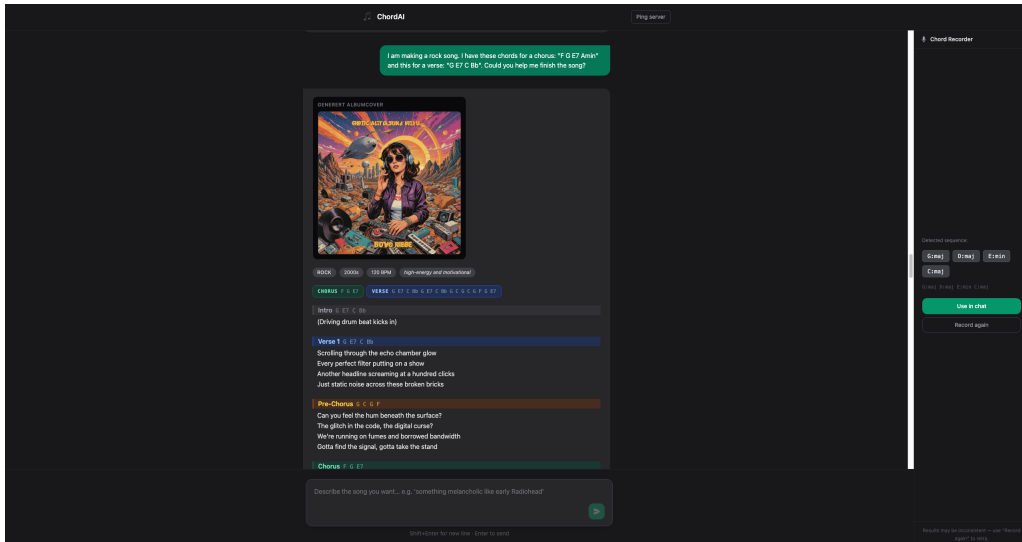


Figure 3: The ChordAI web interface: a chat-style client that shows stage-by-stage generation progress and renders the finished song with per-section chord headers and metadata pills.

7.2 Wiring the real chord model into the server

For most of development the chord runner was a hardcoded stub returning fixed per-genre progressions, which let the rest of the stack be tested before training finished.

Replacing it with a real subprocess call to `sample_chords.py` required several careful choices. The script must be run from the `nanoGPT/` [9] directory, since it parses flags via `exec(open('configurator.py').read())`; the server detects CUDA through `nvidia-smi` and sets `-device` accordingly; and the output parser had to be rewritten, because the original version accidentally matched the diagnostic `Prompt:` line (which contains `<genre:...>`) instead of the generated output — the corrected parser reads the lines immediately following the `sample 1` marker. The stub is retained as a fallback if the script or checkpoint is missing.

7.3 Robustness: seed-chord normalisation and extend mode

Notation crashes. A user-supplied chord in non-standard notation (`F:maj G:maj E:maj A:min`) crashed the sampler with a `KeyError`, because tokens such as `F:maj` are absent from the vocabulary (which uses `F` for F major, `Amin` for A minor, and writes sharps as `s` rather than `#`). We responded with three layers of defence:

- a planner prompt rule that normalises notation at the source;
- a Go `normalizeSeedChords` step that maps `#→s`, strips redundant major suffixes, and rewrites minor notation;
- a graceful skip in the sampler, which now warns and skips any unknown token instead of raising.

In the worst case, skipping all unknown tokens degrades the prompt to genre and decade only — that is, generate mode — a graceful degradation rather than a crash.

Extend mode. Running the chord model in pure extend mode three times produced identical chords for every section, because repeating the seed is the most probable continuation. We therefore use extend mode only for the first run, preserving the user’s seed-section chords, and switch the second and third runs to unconditional generate mode to obtain independent material for the non-seed sections. A related failure — extend mode sometimes emits a flat chord stream with no section tags, leaving the palette empty — is handled by synthesising a `verse` entry from the user’s own (normalised, cycle-detected) seed chords whenever the merged palette would otherwise be empty, guaranteeing that the lyric model always receives at least the chords the user explicitly provided.

7.4 Reproducibility and tooling

For reproducibility, the chord-gen subproject’s build was consolidated behind a single `Makefile` whose order-only prerequisites cause `make train` on a fresh clone to run setup and data preparation first. Dependencies were pinned to their six direct entries, and the pipeline, GPU/MPS instructions, sampling examples, and vocabulary reference were documented in a `README`.

8 Chord Recognition from Audio

To remove the friction of typing chords by hand, the system includes an audio pathway that detects chords from a recording. Its development serves as an instructive engineering narrative, transitioning from an ambitious real-time streaming pipeline to a decoupled, deep learning-driven offline API.

8.1 The Original Real-Time Architecture

The original architecture functioned as a continuous, live-listening system. Audio was captured directly from a live microphone using a command-line recording utility called `parec`. This raw audio stream flowed into a central algorithmic component called the `TurnRouter`. The router

acted as an audio splitter, evaluating the incoming stream simultaneously for speech and musical chords.

For speech detection, it utilized a predictive algorithm that calculates the statistical likelihood of human speech in an audio signal, known as Voice Activity Detection (VAD) [18]. If the VAD score exceeded a specific threshold, the audio chunk was routed into a dedicated speech buffer. For music, it monitored the volume using a mathematical calculation representing the average continuous power of the audio signal, called Root Mean Square (RMS) energy. To prevent erratic volume spikes from acoustic guitar strums from confusing the router, a smoothing technique that applies more weight to recent data points to filter out erratic spikes, known as an Exponential Moving Average (EMA), was applied. If this smoothed volume exceeded preset thresholds, the audio was routed into a chord buffer. Once the system detected a sufficient period of silence, a timeout triggered a flush, sending both buffers downstream.

The flushed bytes then entered the transcription engine. Speech bytes were processed by a local pywhispercpp model used to translate spoken audio into text, an approach known as Automatic Speech Recognition (ASR). Chord bytes were processed mathematically: the system applied a mathematical acoustic transform that maps audio waveforms into specific musical frequencies, called the Constant-Q Transform (CQT) [17]. This data was matched against static chord templates, and the system applied a dynamic programming algorithm that finds the most likely sequence of hidden states, known as Viterbi Decoding [15], to prevent the output from jittering rapidly and to result in stable chord segments. Finally, an agent orchestrator packaged the transcribed text and chords into a structured JSON payload to be sent to a Large Language Model.

8.2 Critical Analysis of the Real-Time Pipeline

The real-time workflow contained critical operational flaws that forced its abandonment. Running a live microphone through a compatibility layer for running Linux binaries on Windows, called the Windows Subsystem for Linux (WSL), introduced severe hardware routing constraints. Because standard soundcards are not natively exposed in this environment, native audio libraries routinely failed to detect devices. Furthermore, Windows-native noise suppression algorithms actively filtered out acoustic guitar frequencies, erroneously treating them as background noise.

The mathematical CQT chord recognition model was also overly rigid. It routinely failed to distinguish between suspended chords that shared identical pitch classes, requiring heuristic bass-bias adjustments to force the algorithm to respect the acoustic root note. Additionally, the **TurnRouter**'s reliance on silence timeouts was fundamentally flawed for musical applications; the long resonant tails of acoustic guitars made volume thresholds finicky, causing the system to frequently cut off recordings prematurely.

8.3 The Offline Deep Learning Pivot

To resolve these systemic issues, the live-listening pipeline was archived as a legacy proof-of-concept, and the system was transformed into a decoupled, API-driven backend.

Instead of capturing live microphone data, the new workflow relies on an independent web server built with FastAPI that accepts pre-recorded audio files. These are submitted as **FormData** via a POST request to the `/verify-chords` endpoint from the Vue/Nuxt frontend. Upon receipt, the FastAPI server saves the upload file to temporary disk storage. Because web browsers record highly compressed audio formats that crash the processing engine, transcoding steps are immediately triggered. External system utilities like FFmpeg are invoked via subprocess to automatically decode, down-mix, and transcode these uploads into a 16kHz mono temporary WAV file.

The chord transcription engine also received a major architectural overhaul. The rigid mathematical template matching was replaced with Deep Learning. The system now utilizes the

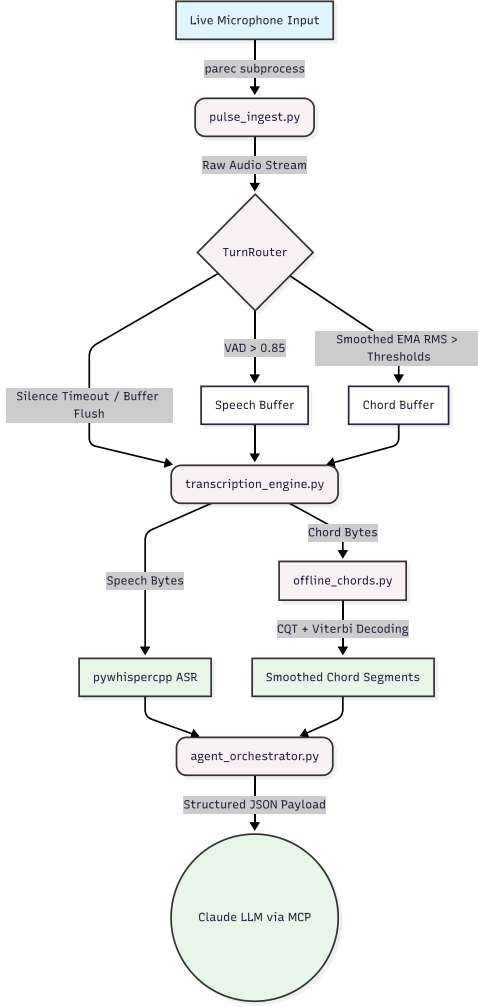


Figure 4: Original real-time streaming architecture.

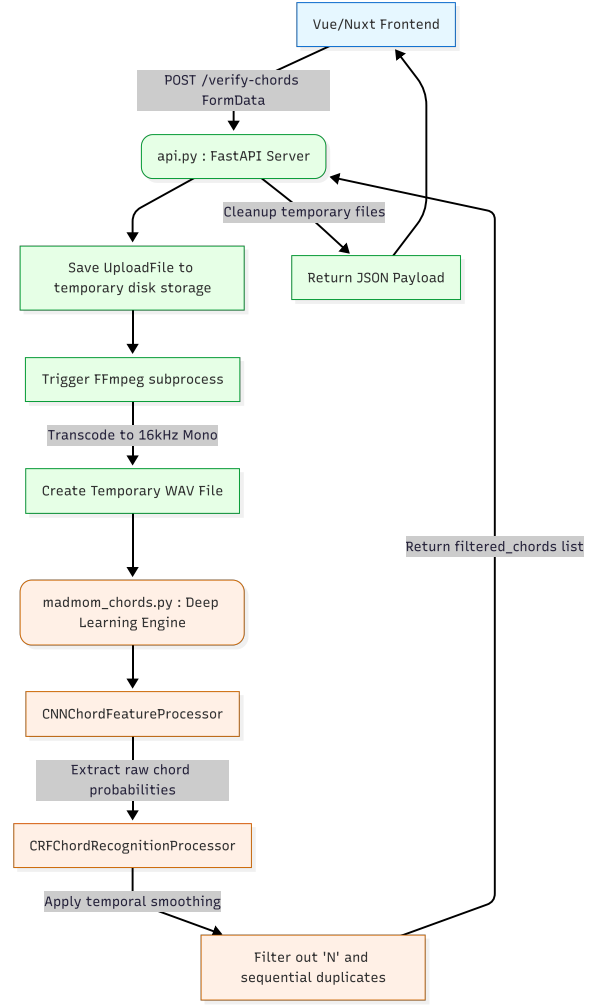


Figure 5: Current offline, API-driven architecture.

madmom library [16], specifically utilizing a deep learning model highly effective at finding spatial and temporal patterns in complex data, known as a Convolutional Neural Network (CNN), to analyze the audio and extract raw chord probabilities. This is followed by a statistical modeling method used to smooth those frame-by-frame predictions into continuous, logical chord segments, called a Conditional Random Field (CRF) [16].

Finally, a post-processing filter is applied to the CRF output to remove any 'N' ("No Chord") segments and filter out sequential duplicates, resulting in a clean, distinct list of chord progressions. This filtered list is packaged into a JSON payload and returned to the frontend. Concurrently, the backend actively cleans up the temporary files from disk. This shift eliminated the accuracy issues caused by the previous mathematical baseline and made the system highly resistant to acoustic noise and performance artifacts.

9 Future Work

For the future work We would like to make the pipeline stages *toggleable* (e.g. turn off the image generator for faster runs, or generate chords only) and to run stages *in parallel* rather than strictly sequentially. On the audio side, a single “record” button that captures the user’s voice and guitar in one clip reviving the dropped speech-to-text alongside chord recognition would close the loop on the original full-input vision.

Chord model. The most valuable next steps follow directly from the limitations in Section 3.9: *multi-seed conditioning evaluation* for formal statistical significance on the v2 > RNN gap; an *optional repetition penalty* at sampling time, documented as an ablation (the high `<eos>` rates at $T=1.0$ suggest looping is already mild); *class-balanced training* with per-genre evaluation for the rare classes (electronic, reggae, jazz); *document-isolation attention* (a block-diagonal mask across `<eos>` boundaries) to complement v2’s token-identity loss masking; and *ordinal decade encoding* so sparse decades can borrow strength from neighbours. More broadly, the chord model is bounded by its dataset: it produces progressions very similar to Chordonomicon, whose quality is itself uneven, so a better or augmented dataset is a natural avenue.

10 Conclusion

ChordAI is a complete, fully-local generative songwriting system: a planner interprets free text and routes it through a custom chord model, a lyric generator, and an album-cover diffuser, with an optional audio pathway for chord input. The engineering, a Go orchestrator, a streaming WebSocket protocol, a Nuxt client, and a robust integration layer, makes the system usable end to end, and the deliberate MCP-to-local pivot kept everything open-source and self-hosted.

The intellectual core, however, is the chord model. By framing three use cases as a single prompt-conditioned autoregressive task and comparing an LSTM, a naive transformer, and an aligned-batching transformer under two complementary metrics, we found something more interesting than a simple architecture ranking. Attention clearly wins on next-chord prediction, especially for harmonically complex genres, but on conditioning faithfulness the naive transformer underperforms the LSTM, because attention’s freedom lets it ignore a conditioning signal the LSTM is forced to carry, and only a corrected training recipe makes the transformer both predict better *and* condition more faithfully. The lesson, that architecture and training recipe are not independent contributors, is the kind of result that only the full three-model, two-metric design could surface, and it is the finding we are most pleased to have uncovered.

References

- [1] The Go Programming Language. (n.d.), The Go Programming Language. Retrieved 17:00, May 31. 2026. from <https://go.dev>
- [2] Gin Web Framework. (n.d.), Gin Web Framework. Retrieved 17:00, May 31. 2026. from <https://gin-gonic.com>
- [3] Vue.js. (n.d.), Vue.js – The Progressive JavaScript Framework. Retrieved 17:00, May 31. 2026. from <https://vuejs.org>
- [4] Nuxt. (n.d.), Nuxt – The Intuitive Vue Framework. Retrieved 17:00, May 31. 2026. from <https://nuxt.com>
- [5] Google DeepMind. (2 Apr 2026), Gemma 4 Model Card. Retrieved 17:00, May 31. 2026. from https://ai.google.dev/gemma/docs/core/model_card_4
- [6] Allen Institute for AI. (Nov 2024), OLMo-2-1124-7B-Instruct. Retrieved 17:00, May 31. 2026. from <https://huggingface.co/allenai/OLMo-2-1124-7B-Instruct>
- [7] Hugging Face. (n.d.), Transformers Documentation. Retrieved 17:00, May 31. 2026. from <https://huggingface.co/docs/transformers>
- [8] Ollama. (n.d.), Ollama. Retrieved 17:00, May 31. 2026. from <https://ollama.com>

- [9] GitHub. (2025), nanoGPT: The simplest, fastest repository for training/finetuning medium-sized GPTs from <https://github.com/karpathy/nanoGPT>
- [10] Kantarelis, S., Thomas, K., Lyberatos, V., Dervakos, E., & Stamou, G. (2024). Chordonomicon: A dataset of 666,000 songs and their chord progressions. *arXiv preprint arXiv:2410.22046*.
- [11] Sauer, A., Lorenz, D., Blattmann, A., & Rombach, R. (2024). Adversarial diffusion distillation. In *European Conference on Computer Vision* (pp. 87–103). Springer, Cham.
- [12] Rombach, R., Blattmann, A., Lorenz, D., Esser, P., & Ommer, B. (2022). High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (pp. 10684–10695).
- [13] von Platen, P., Patil, S., Lozhkov, A., et al. (2022). Diffusers: State-of-the-art diffusion models. *GitHub repository*. <https://github.com/huggingface/diffusers>
- [14] Goodfellow, I., Pouget-Abadie, J., Mirza, M., et al. (2020). Generative adversarial networks. *Communications of the ACM*, 63(11), 139–144.
- [15] Ciaperoni, M., Gionis, A., Katsamanis, A., & Karras, P. (2022). SIEVE: A Space-Efficient Algorithm for Viterbi Decoding. *Proceedings of the 2022 International Conference on Management of Data*, 1136–1145. <https://doi.org/10.1145/3514221.3526170>
- [16] Korzeniowski, F., & Widmer, G. (2016). A fully convolutional deep auditory model for musical chord recognition. *2016 IEEE 26th International Workshop on Machine Learning for Signal Processing (MLSP)*, 1–6. <https://doi.org/10.1109/mlsp.2016.7738895>
- [17] Nisar, S., Khan, O. U., & Tariq, M. (2016). An Efficient Adaptive Window Size Selection Method for Improving Spectrogram Visualization. *Computational Intelligence and Neuroscience*, 2016, 1–13. <https://doi.org/10.1155/2016/6172453>
- [18] Park, J., Kim, W., Han, D. K., & Ko, H. (2014). Voice Activity Detection in Noisy Environments Based on Double-Combined Fourier Transform and Line Fitting. *The Scientific World Journal*, 2014, 1–12. <https://doi.org/10.1155/2014/146040>

A Reproducibility notes and footguns

- **Vocabulary tokens must be whitespace-free.** Genres with spaces are slugified (pop rock→<genre:pop_rock>) so `str.split()` is a valid tokeniser everywhere. Forgetting this caused a 34k-song bug in early iterations.
- **eval_nextchord.py's -val-frac and -seed must match the prepare.py run** that built the data, or the script will evaluate on training songs. Defaults (0.1 / 42) match the project's `prepare.py`.
- **v1 and v2 raw validation losses are not directly comparable** (v2 masks conditioning tokens out of its loss). Use the two evaluation scripts, not raw loss.
- **v2 training requires train.offsets.npy and val.offsets.npy from prepare.py;** `doc_aware_batching=True` errors clearly if they are absent.
- **Sanity check:** step-0 training loss should be $\approx 6.7 \approx \ln(851)$. If not, the vocab/data did not load correctly.

B Quick-start (chord model)

```
# 1. Clean the raw dataset
python clean_chordonomicon.py --input chordonomicon_v2.csv --outdir ./cleaned

# 2. Build nanoGPT binaries (stratified split, offsets, meta.pkl)
python prepare.py --data-dir ./cleaned --outdir ./data/chords

# 3. Train the three models (cluster GPU)
python train.py config/train_chords.py --out_dir=out-CHORDv1 --doc_aware_batching=False
python train.py config/train_chords.py --out_dir=out-CHORDv2 --doc_aware_batching=True
python train_rnn.py --dataset=chords --out_dir=out-CHORDrnn --device=cuda \
    --max_iters=20000 --batch_size=64 --block_size=256

# 4. Generate (temperature ~1.0 recommended)
python sample_chords.py --out_dir=out-CHORDv2 --mode=generate \
    --genre=jazz --decade=2010 --temperature=1.0

# 5. Evaluate the three-way comparison
python eval_nextchord.py out-CHORDrnn out-CHORDv1 out-CHORDv2 \
    --data-dir=data/chords --clean-dir=cleaned --val-frac=0.1 --seed=42
python eval_conditioning.py out-CHORDrnn out-CHORDv1 out-CHORDv2 \
    --data-dir=data/chords --n-per-genre=300 --decade=2010 --temperature=1.0
```

C Using ChordAI application

Demo Video: <https://www.youtube.com/watch?v=VxRHj6AtEQQ>

I have this chorus for a 2000 pop song: F:maj D:min C:maj Bb:maj. Could you help me finish the song

GENERERT ALBUMCOVER



POP

2000s

120 BPM

upbeat and catchy

BRIDGE Dmin C Bb F

CHORUS F Dmin C Bb

INTERLUDE F Dmin C Bb

INTRO Dmin G C F

OUTRO F Dmin C Bb

VERSE F Dmin C Bb

Intro Dmin G C F

We were burning out the midnight hours
Chasing highs on borrowed power
Little secrets in the static glow
Nowhere left for us to go

Verse 1 F Dmin C Bb

Scrollin' through the timeline glow
Every picture staged, a perfect show
Filtered feelings, bite-sized lies
Seen it in a million digital eyes
We build our world on bandwidth lines
Ignoring all the warning signs

Pre-Chorus F Dmin C Bb

Say you got the key to every lock
A time machine to stop the clock
But the battery is draining low
Tell me where this feeling goes

Chorus F Dmin C Bb

Oh, we're running on a fragile sync
A high-voltage, beautiful brink
Yeah, we burn it out in a flash of light
Just another perfect neon night
We were never built for slow decay
We needed this electricity today

Verse 2 F Dmin C Bb

Your status says you're doing fine
Got the perfect caption, the perfect line
But your eyes tell a different truth

Figure 6: Example of a prompt and response using ChordAI.

I am making a groovy punk song. I like the sound of these chords: Dmin G C F. Could you help me finish the song?

GENERERT ALBUMCOVER



PUNK

2000s

160 BPM

energetic and carefree

BRIDGE F G

CHORUS Dmin G C F G F C

INTRO Dmin G C F

OUTRO F G F G F C

VERSE F G F G F C

Intro Dmin G C F

Three-hour screen glow, another night to lose
Scrolling through the ghosts and the things I gotta choose
Yeah, the metrics and the likes, the curated empty show
We're wired to the signal, nowhere left to go

Verse 1 F G F G F C

Another Tuesday night in fluorescent haze
Just talking through the filters of these empty days
The algorithms know me, they predict what I will buy
Another perfect moment that I just pretend is high

Pre-Chorus F G F G

Gotta shake the signal off, gotta break the feed
Need the analog chaos, the raw, forgotten creed

Figure 7: Example of a prompt and response using ChordAI.

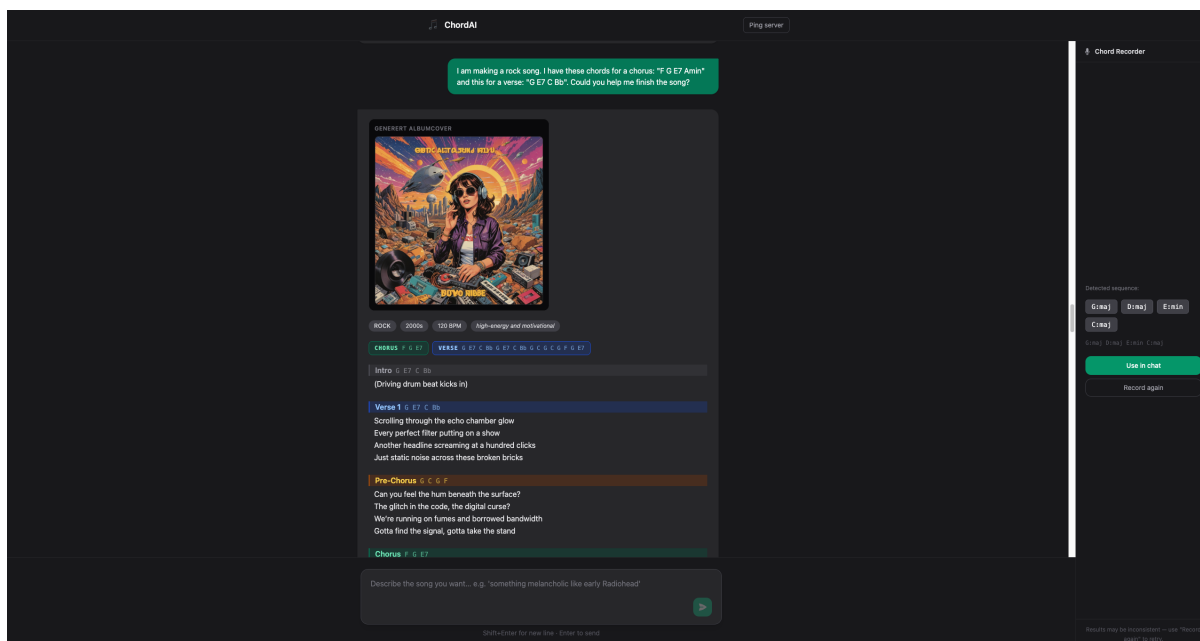


Figure 8: Example of what the ChordAI user interface looks like.

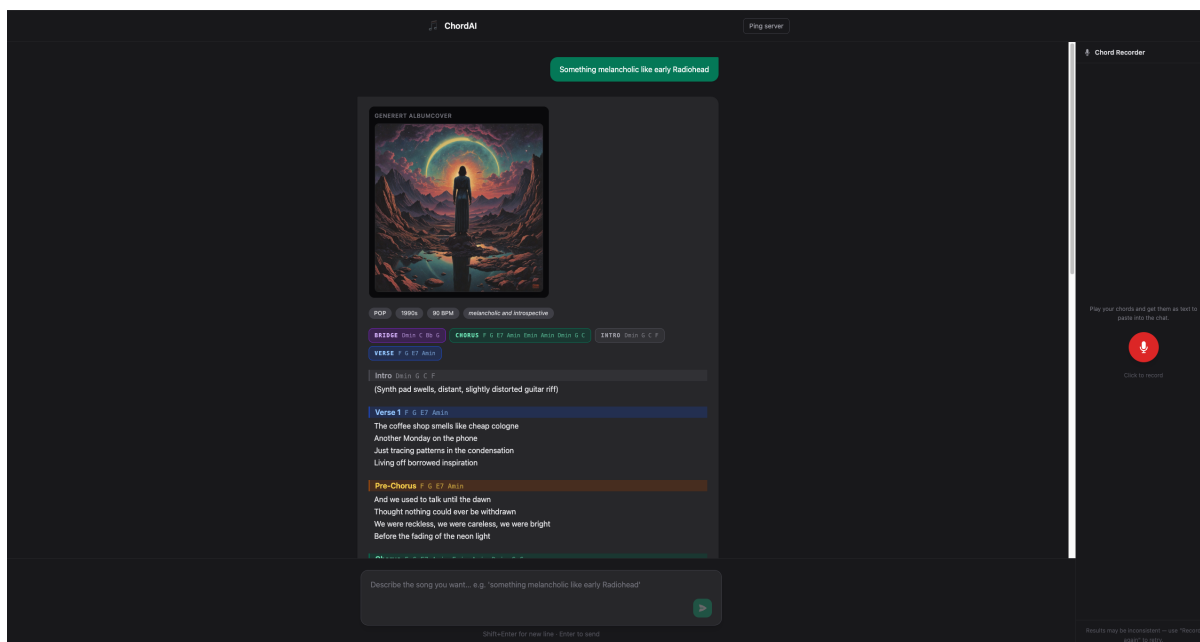


Figure 9: Example of what the ChordAI user interface looks like.